

Lesson 14 — Model Tuning & Hyperparameter Optimization

Now we move from PCA to one of the most important practical ML topics: improving a model by choosing better hyperparameters.

1. What is Model Tuning?

Suppose you train a Random Forest:

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    max_depth=5
)
```

How did we decide:

```
n_estimators = 100
max_depth = 5
```

These values are called hyperparameters.

Model tuning means finding good hyperparameter values that give better performance.

2. Parameters vs Hyperparameters ☆

This distinction is important.

Parameters

Learned automatically by the model from training data.

For example, in Linear Regression:

weights intercept

The model learns them.

Hyperparameters

Chosen before training.

Examples:

```
KNN:
k = 5

Decision Tree:
max_depth = 5

Random Forest:
n_estimators = 100
```

```
SVM:  
C = 1  
  
K-Means:  
n_clusters = 3
```

Suppose a Decision Tree gives:

```
Training accuracy = 99%
Testing accuracy  = 72%
```

The model may be overfitting.

Changing:

max depth

could help.

Another model might have:

```
Training accuracy = 75%
Testing accuracy  = 73%
```

This may indicate the model is too simple.

So hyperparameter tuning helps us find a better balance.

Before tuning, we normally separate our data:

```
from sklearn.model_selection import train_test_split

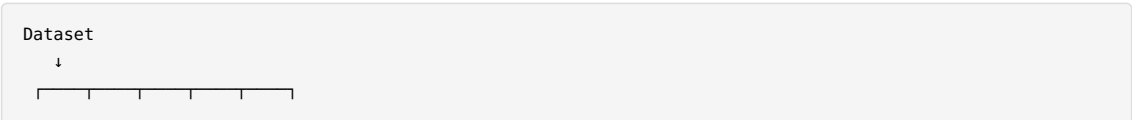
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

This gives:

```
80% → Training
20% → Testing
```

Instead of relying on just one train/test split we can use cross-validation

For example, with 5-fold cross-validation:



F1	F2	F3	F4	F5
----	----	----	----	----

The model trains and validates multiple times.

```
Round 1 → F1 validation
Round 2 → F2 validation
Round 3 → F3 validation
Round 4 → F4 validation
Round 5 → F5 validation
```

Then we calculate the average score.

6. Grid Search

One common tuning method is GridSearchCV.

Suppose we want to test:

```
max_depth = 3, 5, 7
n_estimators = 50, 100, 200
```

Grid Search tests combinations:

```
3 × 50
3 × 100
3 × 200

5 × 50
5 × 100
5 × 200

7 × 50
7 × 100
7 × 200
```

That's:

9 combinations

7. GridSearchCV Code

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    random_state=42
)

param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [3, 5, 7]
}

grid_search = GridSearchCV(
    model,
    param_grid,
    cv=5,
    scoring="accuracy"
)

grid_search.fit(X_train, y_train)
```

Now find the best parameters:

```
print(grid_search.best_params_)
```

And the best cross-validation score:

```
print(grid_search.best_score_)
```

8. Best Model

You can get the best model using:

```
best_model = grid_search.best_estimator_
```

Then evaluate it:

```
accuracy = best_model.score(X_test, y_test)

print("Test Accuracy:", accuracy)
```

9. Randomized Search

Grid Search can become expensive.

Imagine:

```
10 hyperparameters
x
10 possible values each
```

That could create an enormous number of combinations.

Instead, we can use:

RandomizedSearchCV

It randomly tests a specified number of combinations.

```
from sklearn.model_selection import RandomizedSearchCV

random_search = RandomizedSearchCV(
    model,
    param_distributions=param_grid,
    n_iter=10,
    cv=5,
    scoring="accuracy",
    random_state=42
)

random_search.fit(X_train, y_train)
```

Then:

```
print(random_search.best_params_)
```

10. Grid Search vs Random Search

Grid Search	Random Search
Tests specified combinations	Randomly samples combinations
Can be expensive	Usually faster
Good for smaller search spaces	Good for larger search spaces
Exhaustive within grid	Not exhaustive

11. Example: KNN Tuning

For KNN, one important hyperparameter is:

n_neighbors

We can test:

```
param_grid = {  
    "n_neighbors": [3, 5, 7, 9, 11]  
}
```

Then:

```
from sklearn.neighbors import KNeighborsClassifier  
from sklearn.model_selection import GridSearchCV  
  
model = KNeighborsClassifier()  
  
grid = GridSearchCV(  
    model,  
    param_grid,  
    cv=5,  
    scoring="accuracy"  
)  
  
grid.fit(X_train, y_train)  
  
print(grid.best_params_)  
print(grid.best_score_)
```

12. Important Hyperparameters to Know

KNN

- n_neighbors
- weights
- metric

Decision Tree

- max_depth
- min_samples_split
- min_samples_leaf

Random Forest

- n_estimators
- max_depth

- min_samples_split
- min_samples_leaf
- max_features

SVM

- C
- kernel
- gamma

Gradient Boosting

- n_estimators
- learning_rate
- max_depth
- subsample

K-Means

- n_clusters
- n_init

13. Don't Tune on the Test Set ⚠

This is a very important rule.

Don't do:

```
Test set
  ↓
Try many models
  ↓
Choose the best
```

Because you've effectively used the test data to make model decisions.

Instead:

```
Dataset
  ↓
Training data
  ↓
Cross-validation
  ↓
Hyperparameter tuning
  ↓
Best model
  ↓
Final test set evaluation
```

The test set should ideally be used only for final evaluation.

14. Complete Example

Here's a simple complete example using the Wine dataset:

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
```

```

from sklearn.model_selection import GridSearchCV

# Load data
wine = load_wine()

X = wine.data
y = wine.target

# Train/Test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Model
model = RandomForestClassifier(
    random_state=42
)

# Hyperparameters
param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [3, 5, 7, None]
}

# Grid Search
grid_search = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

# Train
grid_search.fit(X_train, y_train)

# Best parameters
print("Best Parameters:")
print(grid_search.best_params_)

# Best CV score
print("\nBest CV Score:")
print(grid_search.best_score_)

# Best model
best_model = grid_search.best_estimator_

# Test score
test_accuracy = best_model.score(
    X_test,
    y_test
)

print("\nTest Accuracy:")
print(test_accuracy)

```

Lesson 14 — Core Mental Model

Think of tuning like this:

```

Choose Model
  ↓
Choose Hyperparameters
  ↓
Try different combinations

```

```
↓  
Cross-Validation  
↓  
Find Best Combination  
↓  
Train Best Model  
↓  
Final Test Evaluation
```

Most important concepts:

- Hyperparameters are chosen before training.
- Parameters are learned during training.
- Cross-validation gives a more reliable estimate during tuning.
- GridSearchCV tests specified combinations.
- RandomizedSearchCV tests randomly selected combinations.
- Don't repeatedly tune using the test set.
- The goal isn't simply the highest training accuracy—it is good generalization to unseen data.

One-line definition:

Hyperparameter optimization is the process of searching for hyperparameter values that produce the best-performing model, usually using cross-validation.